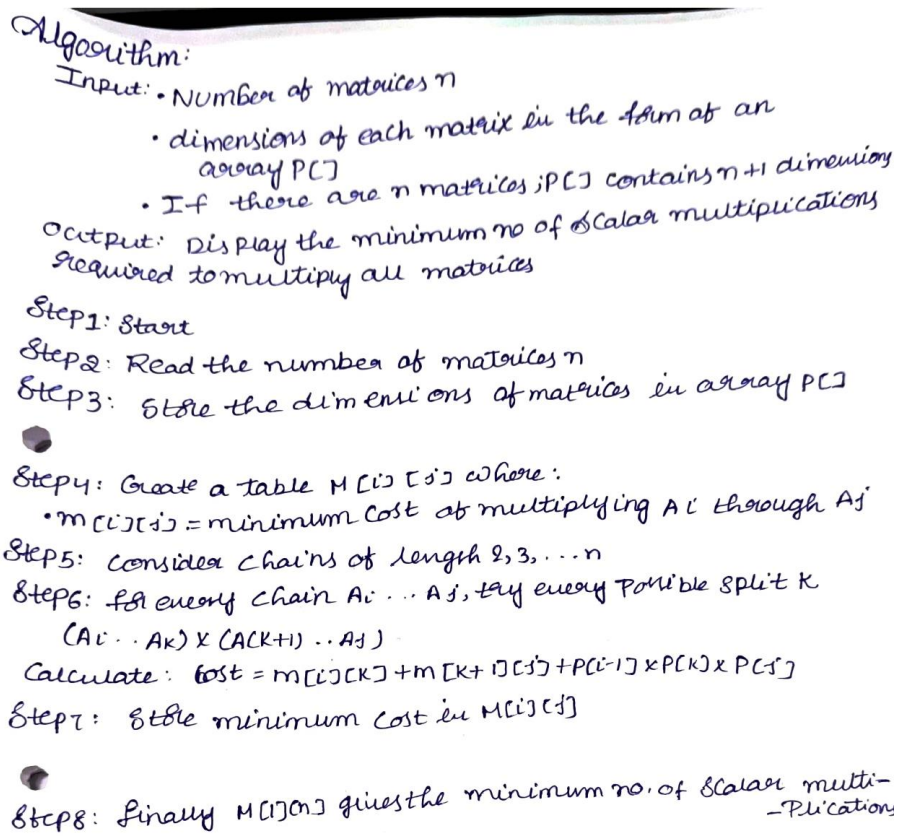
PRACTICAL-6

Aim: To Implementation of chain matrix multiplication using dynamic programming.

Algorithm:



Code:

```
#include

 using namespace std;

int main() { int n;

cout << "Enter number of matrices: ";
cin >> n;

int p[20];

cout << "Enter dimensions of matrices: ";
for (int i = 0; i <= n; i++)
{
   cin >> p[i];
}
```

```
// Display matrices
cout << "\nMatrices:\n";

for (int i = 1; i <= n; i++)
{
   cout << "A" << i << " = [ "
        << p[i - 1] << " x " << p[i]
        << " ]" << endl;
}

int dp[20][20];

// Cost of multiplying one matrix = 0
for (int i = 1; i <= n; i++)
{
   dp[i][i] = 0;
}

// Dynamic Programming
for (int length = 2; length <= n; length++)
{
   for (int i = 1; i <= n - length + 1; i++)
   {
      int j = i + length - 1;

      dp[i][j] = 999999;

      for (int k = i; k < j; k++)
      {
         int cost = dp[i][k]
                + dp[k + 1][j]
                + p[i - 1] * p[k] * p[j];

         if (cost < dp[i][j])
         {
            dp[i][j] = cost;
         }
      }
   }
}

// Display DP Table
cout << "\nDP Table:\n\n";

cout << "       ";
for (int j = 1; j <= n; j++)
{
   cout << "A" << j << "\t";
}
cout << endl;
```

```
for (int i = 1; i <= n; i++)
{
   cout << "A" << i << "     ";

   for (int j = 1; j <= n; j++)
   {
      if (j < i)
         cout << "-\t";
      else
         cout << dp[i][j] << "\t";
   }

   cout << endl;
}

// Minimum cost
cout << "\nMinimum number of scalar multiplications = "
    << dp[1][n] << endl;

return 0;


}
```

Output:

```
PS C:\Desktop\dda-practicals>  g++ practical-6.cpp -o practical-6.exe
PS C:\Desktop\dda-practicals> C:\Desktop\dda-practicals/practical-6.exe
Enter number of matrices: 4
Enter dimensions of matrices: 2
1
4
6
2

Matrices:
A1 = [ 2 x 1 ]
A2 = [ 1 x 4 ]
A3 = [ 4 x 6 ]
A4 = [ 6 x 2 ]

DP Table:

      A1      A2      A3      A4
A1    0       8       36      40
A2    -       0       24      36
A3    -       -       0       48
A4    -       -       -       0

Minimum number of scalar multiplications = 40
PS C:\Desktop\dda-practicals>
```

Time complexity:

Time Complexity:

- for len = 2 to n → n times
- for i = 1 to n-length+1 → n times
- for k = i to j-1 → n times

So approximately:

$n \times n \times n = n^3$

Therefore, Time Complexity = $O(n^3)$ → best case

Loop 1 → length = 2 to n → n
Loop 2 → i = 1 to n - length + 1 → n
Loop 3 → k = i to j - 1 → n
no. of operations = n x n x n => n^3

So, Time complexity is same for all cases
Best case = worst case = average case = n^3

Result:
The Chain Matrix Multiplication problem was successfully implemented using Dynamic Programming. The DP table was constructed to find the minimum number of scalar multiplications required to multiply the given sequence of matrices.